

# ROS 1

מדריך שימושי – V2.0

[תאריך]



מתניה בשירי

פרופ' אמיר שפירא, אונ' בן גוריון – הנדסת מכטרוניקה

**ROS - System Operating Robot** היא פלטפורמה המהווה מסגרת תוכנה (framework) לפיתוח מערכות רובוטיות. למרות השם, היא אינה מערכת הפעלה במובן הקלאסי, אלא אוסף של כלים, ספריות וסטנדרטים המאפשרים תכנון, יישום ופריסה של רובוטים מורכבים באופן יעיל ומודולרי.

אחד היתרונות המרכזיים של ROS הוא המודולריות, המאפשרת שימוש בחבילות קיימות והתאמתן לאפליקציות שונות עם שינויים מינימליים. בנוסף, המערכת מספקת תשתית תקשורת פשוטה בין הרכיבים השונים, ללא צורך בהתעסקות עם פרטוקולים מורכבים. יתרון נוסף הוא קיומן של סביבות סימולציה מתקדמות, כגון **Gazebo**, המאפשרות בדיקות וולידציה של אלגוריתמים לפני פריסה ברובוט פיזי.

עם זאת, בניגוד לפיתוח קונבנציונלי, **העבודה עם ROS מחייבת היכרות עם המעטפת הרחבה של המערכת**. לעיתים, אפילו פעולות פשוטות דורשות הבנה מעמיקה של האופן שבו ROS בנוי ומתנהל, מה שעלול להקשות על מפתחים בתחילת הדרך.

בתחילת דרכי בעולם הרובוטיקה, כשלמדתי לפתח באמצעות ROS נתקלתי באתגר משמעותי של היעדר תמונה רחבה של המערכת. ללא ראייה מערכתית ברורה, מצאתי את עצמי מתקשה להשיג עצמאות וביטחון בפיתוח רובוטים, ואף בנייתו של מערכות קיימות.

לפיכך, החלטתי לרכז ולסכם את המעטפת ואופן הפיתוח בROS, מתוך מטרה להנגיש את ROS למפתחים בתחילת דרכם, ולספק להם מדריך ברור ומקיף ובכך להנגיש את הבנת הפלטפורמה ולפנות את המפתח להתמקד בפיתוח הרובוט עצמו.

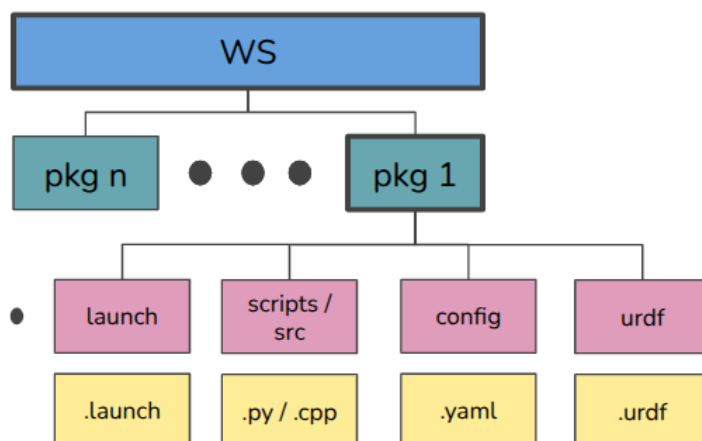
המדריך המובא לפניכם, מחולק לשני חלקים מרכזיים:

**הבנה תאורטית** ובו יתואר מבנה ההיררכי של פרויקט ב-ROS, חלוקה לתיקיות וסוגי המסמכים הייעודיים לכל תיקיה ושיטות העברת המידע. **החלק השני יעסוק במדריך מעשי** ויצג שלב אחרי שלב פיתוח רובוט פשוט יחסית מתוך מטרה להציג את סדר הפעולות האופייניות הכרת מבנה הקוד בכל אחד מהקבצים השונים – ובכך יובן באופן מעשי כיצד לפתח רובוט מ0.

בהצלחה

## 1. המבנה ההיררכי של פרויקט ב-ROS.

כל פרויקט ב-Ros ניתן לתאר כמבנה ההיררכי כפי המוצג באיור הבא:



- פרויקט (workspace) – תיקיה מרכזית המכילה את כל הפרויקט מקצה לקצה.
- חבילה (package) – "חבילת פיתוח" אשר תכיל את כל המעטפת הנדרשת.
- תיקיות וקבצים – תכולת הפרויקט, בהם יפותחו כל הנדרש לפרויקט (מידול מכני, חיישנים, מנועים, בקרים, סביבת הסימולציה, פרמטרים, מקטעי קוד של אלגוריתמיקה, קבצי הרצה מרוכזים, פורמטים של data שיעברו בתקשורת ועוד)
- קבצים ותיקיות ל"בניה והדרה" - קבצים המיועדים ל"בניית" הפרויקט והחבילה. חלקם מוגדרים אוטומטית וחלקם נדרש לעדכן באופן ידני כפי שיוצג בהמשך.

## 2. קבצים ותיקיות

הpackage, אבן הבניין המרכזי בפרויקט - מורכב מאוסף תיקיות בעלות שם ותפקיד מאוד מוגדר והן מכילות סוגי קבצים בעלי סיומות מסוג ספציפי התואם את תפקיד התיקיה. באיור הבא תוצג מפת התיקיות בפרויקט סטנדרטי באופן מפורט:

```
my_ws/
├─ build/          ─ catkin_make (נוצרת ע"י) תיקיית בנייה
├─ devel/          ─ setup.bash קבצי הפיתוח, כולל
├─ src/            ─ (packages) כאן שמים את כל החבילות
│  └─ my_package/  ─ (package) חבילה
│     ├─ CMakeLists.txt ─ קובץ הגדרות קומפילציה
│     ├─ package.xml   ─ מידע על החבילה ותלויות
│     ├─ launch/       ─ להפעלת פרויקטים .launch קבצי
│     ├─ urdf/         ─ (URDF / XACRO) קבצי תיאור הרובוט
│     ├─ config/       ─ (PID, בקורות כמו) קבצי תצורה
│     ├─ scripts/      ─ (nodes) סקריפטים בפיתוח
│     ├─ src/          ─ אם יש ++C קוד ב"
│     └─ msg/         ─ הגדרות הודעות מותאמות אישית (אם יש)
```

בטבלאות הבאות יתוארו התיקיות והקבצים בצורה מפורטת ע"פ החלוקה לנפוצים – כאלו שנפגוש ברוב הפרויקטים, ולתוספות – כאלו שנשתמש בהם על פי הצורך:

### נפוצים:

| folder   | files                  | description                                                                    |
|----------|------------------------|--------------------------------------------------------------------------------|
| /urdf    | urdf / .<br>urdf.xacro | בקבצים אלו יתוארו: המודל המכני של הרובוט, המנועים, החיישנים.                   |
| /config  | yaml.                  | קבצי הגדרות ופרמטרים. כדוג: PID, controllers, פרמטרים.                         |
| /scripts | py.                    | סקריפטים ב-Python (לוגיקה ואלגוריתמיקה)                                        |
| /src     | cpp.                   | סקריפטים ב-C++ (לוגיקה ואלגוריתמיקה)                                           |
| /launch  | launch.                | קבצי הרצה מרוכזת. בד"כ נהוג להריץ: עולם, מודל רובוט, בקרים/פרמטרים, node וכו'. |

תוספות ע"פ הצורך (כתלות באפליקציה):

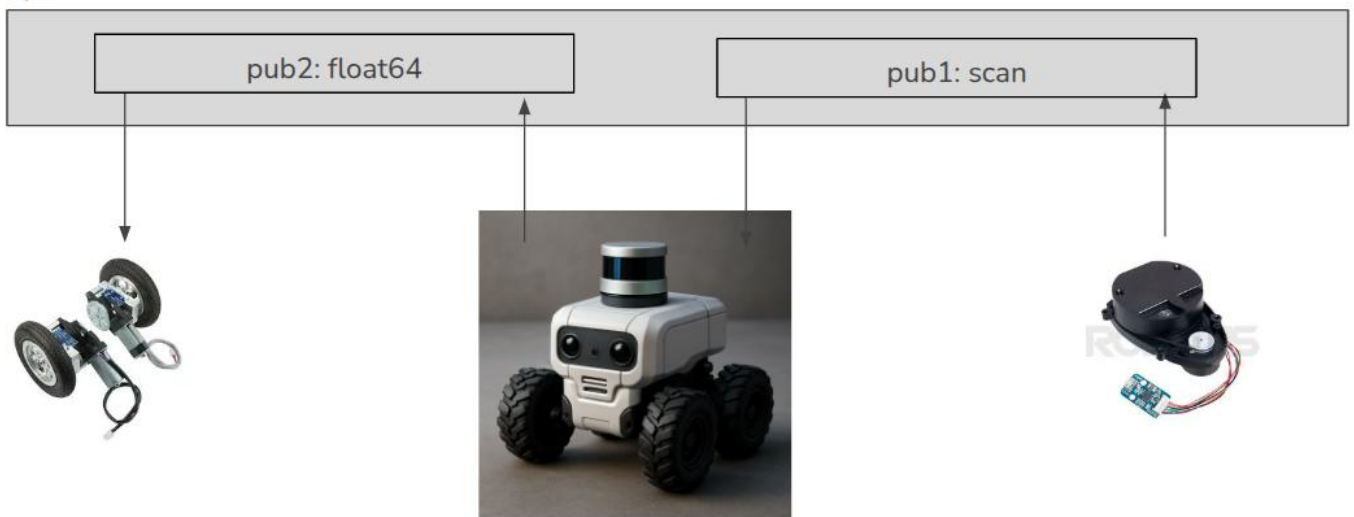
| folder  | files                         | description                                                               |
|---------|-------------------------------|---------------------------------------------------------------------------|
| /msg    | msg.                          | ליצירת פורמט הודעות ל topic.                                              |
| /srv    | srv.                          | ליצירת פורמט הודעות ל service.                                            |
| /action | act.                          | ליצירת פורמט הודעות ל action.                                             |
|         |                               |                                                                           |
| /worlds | world.                        | קבצי סביבה ל-Gazebo (.world).                                             |
| /models | sdf, .config,<br>'<br>meshes. | מודלים תלת-ממדיים לסביבת הסימולציה ב-Gazebo                               |
| /rviz   | rviz.                         | קבצי תצוגה של Rviz (.rviz) – פריסה גרפית של חיישנים, מפרקים, מצלמות וכו'. |

### 3. העברת מידע

כחלק מפיתוח הרובוט נדרש להעביר מידע בין הרכיבים השונים. בטבלה הבאה מוצגים סוגי עברת המידע השונים ומהם ייעודם:

| שיטה | שם      | העברת מידע                 | שימוש נפוץ                     | דו-כיווני? | זמן אמת? |
|------|---------|----------------------------|--------------------------------|------------|----------|
| 1    | Topic   | שידור (Publish/Subscribe)  | חיישנים, מצב מפרקים, מצלמות    | לא         | כן       |
| 2    | Service | בקשה-תגובה (Call/Response) | קבלת מידע או הפעלת פעולה בודדת | כן         | לא       |
| 3    | Action  | פעולה רציפה עם משוב        | תנועה מתמשכת, ניווט            | כן         | כן       |

topic:

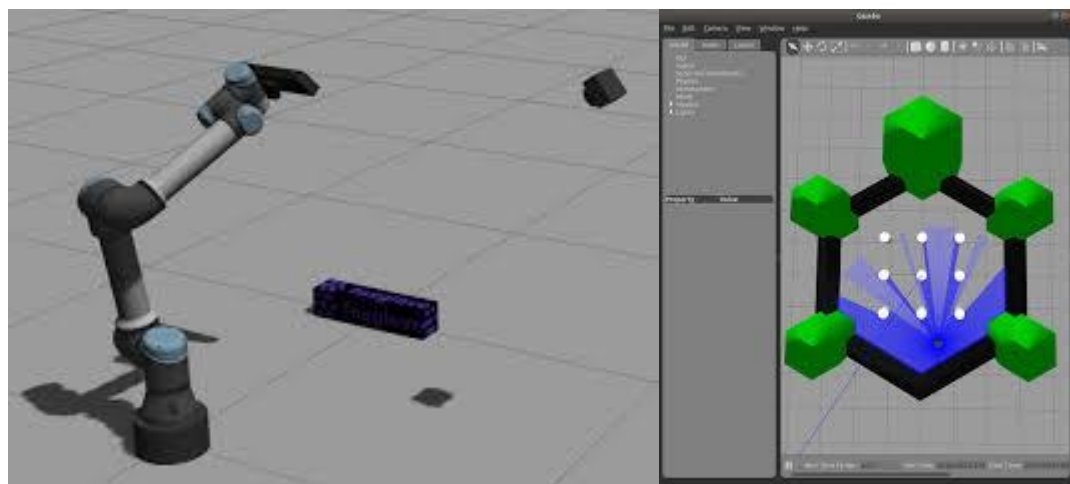


#### 4. תוכנות עזר

כחלק מפיתוח הרובוט נשתמש בכלי תוכנה המיועדים לשימושים שונים:

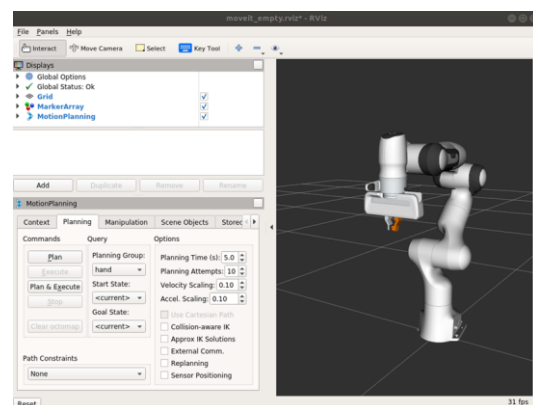
##### GAZEBO:

תוכנת סימולציה המציגה את הרובוט בסביבת "עולם" שיכולה להיות מורכבת מגופים שונים. תוכנה זו גם כוללת בתוכה את הפיזיקה והדינמיקה ובכך ניתן לבחון את הרובוט ללא תלות ברובוט פיזי.



##### RVIZ:

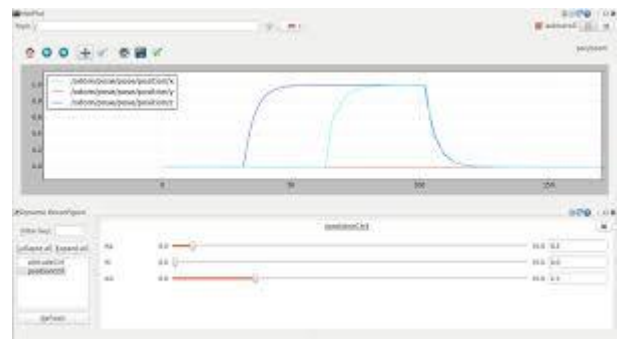
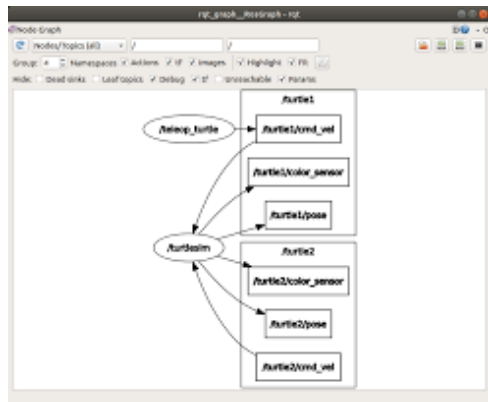
כלי להדמיית הרובוט. שימושי להדמיית הנתונים המתקבלים מחיישנים מצלמות וכדו



RQT:

ממשק עזר לניהול נתונים ב-ROS (ניתוח נתונים, גרפים, הגדרת קבועי בקרה ועוד)

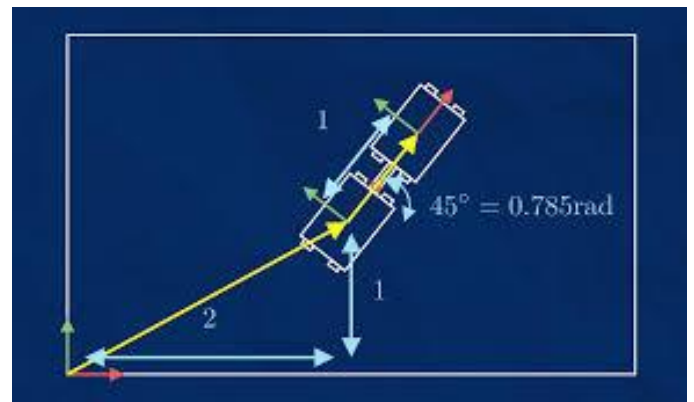
ממשק עזר לניהול נתונים ב-ROS (ניתוח נתונים, גרפים, הגדרת קבועי בקרה ועוד)



TF:

כלי להצגת הצירים של הרכיבים השונים לצורכי חישובי דינמיקה של רכיבים

כלי להצגת הצירים של הרכיבים השונים לצורכי חישובי דינמיקה של רכיבים





## 1. מבוא

בפרק הבא נבנה צעד אחר צעד רובוט פשוט המורכב מקופסא עם 4 גלגלים. ההנעת הרובוט תתבצע בשיטת `diff_drive` באמצעות שני הגלגלים האחוריים, ואילו יחובר סורק מסוג Lidar אשר יסרוק את הסביבה באמצעות מד מרחק לייזר לאורך קשת של 180 מעלות. כפי שצוין לעיל, מטרת פרק זה הוא הכרת flow הפיתוח, מבנה ותכולת קבצי הפיתוח, הכרת קבצי ה"בניה וההדרה", וכן תזמון פעולות "בניה וההדרה".

## 2. FLOW הפיתוח

- יצירת פרויקט חדש (ws)
- "בניה והדרה"
- יצירת חבילה (package)
- "בניה והדרה"
- הוספת קבצים רלוונטיים:
- urdf: מודל רובוט, חיישנים, מנועים.
- yaml: פרמטרים וקבועים
- py: לוגיקה (node)
- launch: קבצי הרצה מרכזיים
- עדכון: CmakeList.txt, package.xml
- "בניה והדרה"
- (הרשאת הרצה - לקבצי .py)
- הרצה (roslaunch, rosrund)
- (מתן פקודות ידניות: rostopic)

---

## הקמת סביבת עבודה בסיסית (WS + PACKAGE)

---

### הקמת פרויקט - WS

בשלב בראשון נייצר תיקיה בשם "my\_ws", ולאחר מכן כאשר "נבנה" אותה מותאמת לROS היא תוגדר כתיקיית ws:

```
mkdir -p ~/my_ws/src
cd ~/my_ws/
catkin_make

source devel/setup.bash
```

---

### הקמת "PACKAGE"

לאחר מכן, נקים "חבילת פיתוח" בשם "pkg\_robotcar" שבמקרה שלנו תכלול את כל הפרויקט מקצה לקצה.

בחבילה שלנו נגדיר בתור התחלה שנרצה להשתמש ב"הודעות סטנדרטיות", קוד בpython וקוד ב ++C.

```
cd ~/my_ws/src
catkin_create_pkg pkg_robotcar std_msgs rospy roscpp
```

ואז נעשה שוב "בניה" לפרויקט:

```
cd ~/my_ws
catkin_make
```

---

### הרחבה על "בניה והדרה"

בהתאם לכל תוכנה סטנדרטית גם Ros נדרש לעשות "build" אחרי כל שינוי. ב Ros ישנו 2 סוגי build:

catkin\_make ו-source devel/setup.bash. לכל אחד תפקיד שונה, אך בד"כ נדרש לעשות רק את catkin\_make וכשניצור פרויקט חדש \ נפתח טרמינל חדש – נדרש גם source devel/setup.bash.

| למה זה חשוב?                 | מתי נדרש?                              | פעולה                   |
|------------------------------|----------------------------------------|-------------------------|
| לבנות את הקוד שלך            | אחרי שינוי בקוד / הודעות / יצירת חבילה | catkin_make             |
| להגדיר את סביבת העבודה ל-Ros | אחרי build / בטרמינל חדש               | source devel/setup.bash |

באותה הזדמנות חשוב להדגיש כי פעולות אלו לא נדרשות לעשות כאשר רק עושים שינוי בתוכן הקובץ. בשורות הבאות ניתן לראות כיצד chat-gpt התייחס למתי נדרש לעשות catkin\_make

### מתי צריך להריץ catkin\_make? ✓

כן - יש להריץ catkin\_make כאשר:

1. נוסף קובץ קוד חדש (C++):

- לדוגמה: הוספת node.cpp לתיקיית src.
- למחצה: כי צריך לקמפל (לבנות) אותו לקובץ הרצה.

2. ערכת את CMakeLists.txt או package.xml:

- כמו הוספת תלות חדשה או שינוי הגדרות בנייה.

3. הוספת קובצי הודעות (msg.) או שירותים (srv.):

- כי נדרש קימפול אוטומטי לקוד.

### מתי לא צריך להריץ catkin\_make? ❌

לא נדרש catkin\_make כאשר:

1. אתה עובד עם סקריפט Python בלבד:

- לדוגמה: יצרת קובץ my\_node.py בתיקיית ./scripts.
- אבל חשוב לוודא שהקובץ עם הרשאות הרצה.

Edit ✎ Copy 📄

bash

```
chmod +x my_node.py
```

2. הוספת קבצי URDF, launch, YAML, config וכו':



- אלה קבצים שנקראים בזמן ריצה, אין צורך לקמפל אותם.

בשלב הבא, אנו נבנה את המודל הפיזי של הרובוט. פעולה זו תתבצע בקובץ `urdf`.

הקובץ המרכזי בו אנו בונים את המודל הפיזי של הרובוט ואת אלמנטי הבקרה (חיישנים ומנועים) הוא `urdf`. במהלך הפרק הבא נראה את שלבי בניית קובץ זה, המסגרת של קובץ (פתיח, סגירה וכדו'), השדות הרלוונטיות לכל סוג (וכן אילו אופציות נוספות עוד קיימות).

בטרם נתחיל ניתן הקדמה קטנה על "אבני הבניין" בקובץ זה:

- `link`: צורה פיזית כדוגמת מרובע, עיגול וכדו'
- `joint`: חיבור ("מפרק") בין 2 `link`'ים. החיבור יכול להיות, קשיח, סיבובי, לינארי וכדו'
- `Plugin`: רכיב בקרתי – מנועים וחיישנים לסוגיהם.

1. בשלב הראשון נייצר תיקיה בשם `urdf` ולאחר מכן נייצר קובץ בשם `robotcar.urdf`.

```
mkdir -p ~/my_ws/src/pkg_robotcar/urdf
cd ~/my_ws/src/pkg_robotcar/urdf
touch robotcar.urdf
```

(כאמור, כאן לא נדרש לעשות `catkin_make`)

2. נבנה את "מסגרת הקובץ"

```
<?xml version="1.0"?>
<robot name="robotcar">
```

```
</robot>
```

```
<!-- בסיס הרובוט - קובייה -->
<link name="base_link">
  <visual>
    <geometry>
      <box size="0.2 0.3 0.1"/>
    </geometry>
    <material name="blue">
      <color rgba="0 0 1 1"/>
    </material>
  </visual>
  <collision>
    <geometry>
      <box size="0.2 0.3 0.1"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="1.0"/>
    <inertia ixx="0.01" ixy="0.0" ixz="0.0"
              iyy="0.01" iyz="0.0"
              izz="0.01"/>
  </inertial>
</link>
```

3. נבנה את מודל הפרויקט:

3.1 תחילה נבנה את קופסת הבסיס:

### נסביר את מקטע הקוד:

- שם הצורה base\_link (link):

שדות עיקריות:

- visual – ממדי הצורה:
  - box size – מידות (בקופסא, בגליל \ כדור זה נראה אחרת)
  - material / color – צבע (שם, וערכי rgb מ 0-1)
- collision – התנגשות: בד"כ דומה לvisual אא"כ יש דרישות אחרות
- inertial – נתוני פיזיקה של החומר:
  - mass: משקל לקוב (צפיפות החומר)
  - inertia: מטריצת מומנטי האינרציה

### 3.2 נבנה את הגלגלים

הגלגלים יבנו כ גליל ("cylinder") ברדיוס  $r=0.1\text{m}$  ובעובי:  $l=0.05\text{m}$ .

```
<!-- גלגלים -->
<!-- גלגל ימני אחורי -->
<link name="rear_right_wheel">
  <visual>
    <geometry>
      <cylinder radius="0.1" length="0.05"/>
    </geometry>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <material name="black">
      <color rgba="0 0 0 1"/>
    </material>
  </visual>
</link>
```

נשים לב שתחת שדה visual התווסף עוד תת שדה "origin" - מנח, כלומר, כיצד יש להזיז את הצורה ביחס למרכז הצירים. כאן הזזנו ב  $90^\circ$  סביב ציר הא (כדי שיעמוד כגלגל ולא ישכב כמו צלחת).

### 3.3 נחבר את הגלגל לקופסא (joint)

```
<!-- חיבור סיבובי לגלגל אחורי ימין -->
<joint name="rear_right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="rear_right_wheel"/>
  <origin xyz="-0.1 -0.15 -0.05" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
</joint>
```

השדות הרלוונטיים:

- שם החיבור – מעבר לצורך לקרוא לכל חלק בשם, נדרש להשתמש בשם זה כשנחבר אותו למנוע.
- Type: סוג החיבור. במקרא שלנו הוא מוגדר כסיבובי(continuous). בטבלה הבאה מוצג כלל סוגי החיבור:

| ✓ כל סוגי ה־joint: |                                  |                          |                                                         |
|--------------------|----------------------------------|--------------------------|---------------------------------------------------------|
| סוג                | תיאור                            | תנועה מותרת              | הערות                                                   |
| fixed              | חיבור קבוע ללא תנועה             | אין תנועה                | משמש למשל לקיבוע של מצלמה או LIDAR לבסיס                |
| revolute           | מפרק מסתובב עם גבולות (כמו מרפק) | סיבוב סביב ציר עם מגבלות | חייב לכלול <limit> (min/max)                            |
| continuous         | מפרק מסתובב ללא גבולות           | סיבוב אינסופי (כמו גלגל) | אין <limit> (לדוגמה: סרוו שמסתובב חופשי)                |
| prismatic          | תנועה קווית (קדימה-אחורה)        | תנועה קווית עם מגבלות    | חייב לכלול <limit>                                      |
| planar             | תנועה במישור XY + סיבוב סביב Z   | תנועה בתוך מישור         | שימושי ל־2.5D סימולציות                                 |
| floating           | חופש מלא (6 דרגות חופש)          | תנועה חופשית בכל הצירים  | נדיר, משמש רק למצבים מיוחדים (כמו רחפן/סימולציה חופשית) |

- parent/child – איזה חלק מתחבר למי. כשתהיה תנועה – ה child ינוע ביחס לparent.
- Origin - הזזה וסיבוב מרכז המסה של ה child למיקום שהוא ביחס למרכז המסה של ה parent.
- Axis – ציר הסיבוב במפרק. מוגדר ביחס ל parent.

3.4 כעת נעשה זאת ליתר הגלגלים

```
<!-- גלגל שמאלי אחורי -->
<link name="rear_left_wheel">
  <visual>
    <geometry>
      <cylinder radius="0.1" length="0.05"/>
    </geometry>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <material name="black"/>
  </visual>
</link>

<joint name="rear_left_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="rear_left_wheel"/>
  <origin xyz="-0.1 0.15 -0.05" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
</joint>
```

```

<!-- 'גלגל ימני' קדמי -->
<link name="front_right_wheel">
  <visual>
    <geometry>
      <cylinder radius="0.1" length="0.05"/>
    </geometry>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <material name="black"/>
  </visual>
</link>

<joint name="front_right_wheel_joint" type="fixed">
  <parent link="base_link"/>
  <child link="front_right_wheel"/>
  <origin xyz="0.1 -0.15 -0.05" rpy="0 0 0"/>
</joint>

```

```

<!-- 'גלגל שמאלי' קדמי -->
<link name="front_left_wheel">
  <visual>
    <geometry>
      <cylinder radius="0.1" length="0.05"/>
    </geometry>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <material name="black"/>
  </visual>
</link>

<joint name="front_left_wheel_joint" type="fixed">
  <parent link="base_link"/>
  <child link="front_left_wheel"/>
  <origin xyz="0.1 0.15 -0.05" rpy="0 0 0"/>
</joint>

```

3.5 הוספת אלמנטי בקרה למודל:

### 3.5.1 הוספת חיישן

שים לב להוספת הצורה (link) ולחיבורו לגוף הרובוט (joint) בצורה קשיחה (fixed)

```
<!-- למעלה LidAR חיישן -->
<link name="lidar_link">
  <visual>
    <geometry>
      <cylinder length="0.05" radius="0.05"/>
    </geometry>
    <material name="black"/>
  </visual>
</link>
<joint name="lidar_joint" type="fixed">
  <parent link="base_link"/>
  <child link="lidar_link"/>
  <origin xyz="0 0 0.1" rpy="0 0 0"/>
</joint>
```

למנוע - לא נדרש להוסיף מודל פיזי

3.6 הוספת הגדרות לאלמנטי הבקרה במידה ואנו מעוניינים בסימולציה:

כאשר אנו רוצים לעבוד בסימולציה אנו צריכים להגדיר את החיישן כיצד הוא יעבוד.

זה המקום לחדד מה קורה כשעובדים עם רובוט בעולם האמיתי ולא בסימולציה. באלמנט אמיתי, כל ההגדרות שאותם נראה בהמשך מוגדרים כבר בדרייבר. הדרייבר יתחבר למחשב באמצעות כבל usb או ethernet, הוא יתפקד כnode (אישות ב-Ros אשר מבצעת לוגיקה, מקבלת \ שולחת מידע וכדו' כפי שיורחב בהמשך), ומפרסם את הdata כtopic. כל מה שצוין כאן מוגדר כתחליף לdriver עצמו.



### 3.6.1 הגדרות "דרייבר" לחיישן lidarn (לסימולציה)

```
<gazebo reference="lidar_link">
  <sensor type="ray" name="lidar">
    <update_rate>10.0</update_rate>
    <pose>0 0 0 0 0 0</pose>
    <ray>
      <scan>
        <horizontal>
          <samples>360</samples>
          <resolution>1</resolution>
          <min_angle>-3.14</min_angle>
          <max_angle>3.14</max_angle>
        </horizontal>
      </scan>
      <range>
        <min>0.1</min>
        <max>10.0</max>
        <resolution>0.01</resolution>
      </range>
      <noise>
        <type>gaussian</type>
        <mean>0.0</mean>
        <stddev>0.01</stddev>
      </noise>
    </ray>
    <plugin name="gazebo_ros_laser_controller" filename="libgazebo_ros_laser.so">
      <topicName>scan</topicName>
      <frameName>lidar_link</frameName>
    </plugin>
  </sensor>
</gazebo>
```

השדות הם די אינטואיטיביות.

בטבלה המופיעה מטה מוצגים נוספים של חיישנים. במידת הצורך ניתן לקחת דוגמא פשוטה מהאינטרנט ולהגדירה בהתאם לצורך.

סוגי חיישנים שניתן להוסיף ב-Gazebo 🔑

| סוג חיישן            | =type        | תיאור                          | Topic טיפוס             |
|----------------------|--------------|--------------------------------|-------------------------|
| LiDAR (Laser)        | ray          | חיישן סריקה לייזר (כמו Hokuyo) | scan/                   |
| מצלמה רגילה          | camera       | מצלמה עם תמונה RGB             | camera/image_raw/       |
| מצלמת עומק (Depth)   | depth        | תמונה עם ערכי עומק לכל פיקסל   | camera/depth/image_raw/ |
| IMU (מד תאוצה וג'יר) | imu          | מד מהירות זוויתית ותאוצה       | imu/data/               |
| GPS                  | gps          | קואורדינטות גלובליות           | gps/fix/                |
| Contact Sensor       | contact      | חיישן מגע / התנגשות            | contact_sensor/         |
| Force/Torque Sensor  | force_torque | מדידת כוחות/מומנטים            | wrench/                 |
| Magnetometer         | magnetometer | מד שדה מגנטי                   | magnetometer/           |
| Altitude             | altimeter    | מד גובה לפי לחץ אוויר          | altimeter/              |

3.6.1 הגדרות דרייבר למנוע מסוג diff\_drive (לסימולציה)

```
<gazebo>
  <plugin name="diff_drive_controller" filename="libgazebo_ros_diff_drive.so">
    <ros>
      <namespace>/</namespace>
      <remapping>cmd_vel:=cmd_vel</remapping>
    </ros>

    <left_joint>left_wheel_joint</left_joint>
    <right_joint>right_wheel_joint</right_joint>

    <wheel_separation>0.4</wheel_separation>    <!-- מרחק בין הגלגלים -->
    <wheel_diameter>0.2</wheel_diameter>        <!-- קוטר הגלגלים -->

    <torque>5.0</torque>
    <command_topic>cmd_vel</command_topic>
    <odometry_topic>odom</odometry_topic>
    <odometry_frame>odom</odometry_frame>
    <robot_base_frame>base_link</robot_base_frame>
    <update_rate>50.0</update_rate>
  </plugin>
</gazebo>
```

גם כאן השדות הם די אינטואיטיביות.

בטבלה המופיעה מטה מוצגים סוגים נוספים של סוגי הנעה. במידת הצורך ניתן לקחת דוגמא פשוטה מהאינטרנט ולהגדירה בהתאם לצורך.

| סוג הנעה                                     | תיאור                                               | יתרונות                     | חסרונות                          |
|----------------------------------------------|-----------------------------------------------------|-----------------------------|----------------------------------|
| <b>diff_drive</b> (דו־גלגלי)                 | הנעה על שני גלגלים אחוריים + גלגל אחד/שניים מייצבים | פשוט, זול, פופולרי          | קשה להסתובב במקום (לא holonomic) |
| <b>skid_steer</b>                            | כל 4 הגלגלים מסתובבים יחד בצדדים (כמו טנק)          | חזק, שטח קשה                | נשחק מהר, החלקה                  |
| <b>mecanum</b>                               | 4 גלגלים עם רולים אלכסוניים                         | יכול לנוע הצידה (holonomic) | מורכב יותר לשליטה                |
| <b>omnidirectional</b> (גלגלים עגולים־מלאים) | 3-4 גלגלים ב־120°                                   | שליטה חופשית במרחב          | מורכב ליישום                     |
| <b>Ackermann</b>                             | כמו רכב – גלגלים קדמיים פונים                       | ריאליסטי, יציב              | קשה לשלוט קרוב למכשולים          |
| <b>legged robots</b>                         | הנעה עם רגליים (Spot, Atlas וכו')                   | עבירות גבוהה מאוד           | יקר ומורכב                       |

## קובץ YAML: הגדרת קבועים ופרמטרים

הקובץ הבא שאותו נלמד הוא קובץ yaml המיועד להגדרת פרמטרים. הסיבות הנפוצות להגדרת פרמטרים הינם: הגדרה וניהול קבועי בקרים, הגדרת הבקרים עצמם (בחומרה אמיתית לא בסימולציה) הגדרת קבועים ברחבי הקוד (כדוג',  $\pi$ , radius\_wheel וכדו'), וקבועים במערכות מורכבות – כך שיש מקום מוגדר לשנותם במקום להתערב בקוד אוו לקמפל מחדש.

במקרה שלנו צורת העבודה תתבצע כך:

1. נייצר תיקיה בשם config ונייצר קובץ בשם my\_robot\_controllers.yaml

```
mkdir -p ~/my_ws/src/pkg_robotcar/config
```

```
touch ~/my_ws/src/pkg_robotcar/config/my_robot_controllers.yaml
```

2. הגדרת הקובץ my\_robot\_controllers.yaml

כפי הצגנו בהתחלה, בסימולציה לא נדרש להגדיר את הבקרים, השדות הללו כבר מוגדרים ב"דרייבר" בקובץ urdf. ולכן כל האמור בפרק זה מתייחס רק עבודה מול חומרה אמיתית.

יש לציין, שבמידה ורוצים להגדיר פרמטרים רוחביים (כמו קבועים פרויקטלים, קבועי PID ועוד) אותם כן נציין בקובץ זה.

```
# מפרסם את מיקום כל המפרקים
joint_state_controller:
  type: joint_state_controller/JointStateController
  publish_rate: 50

# diff_drive בקר של מנוע
diff_drive_controller:
  type: "diff_drive_controller/DiffDriveController"
  left_wheel: left_wheel_joint
  right_wheel: right_wheel_joint
  publish_rate: 50
```

מעטפת הקובץ: אין מבנה מיוחד הדורש תחיליות לסופיות כלשהם.. כל מה שנדרש זה להגדיר בצורת "עץ" את הפרמטרים שברצונך להגדיר.

בקובץ שלנו, נגדיר את הפרמטרים הבאים:

- בקר
  - שם: <משהו שנבחר>
  - סוג: diff\_drive
  - הגדרת הjoint שיחוברו כגלגל ימין וכגלגל שמאל
- משוב

את סוגי הבקרים\משובים השונים ניתן לראות בטבלה הבאה:

| שימוש           | תיאור                                    | type של בקר                                  |
|-----------------|------------------------------------------|----------------------------------------------|
| זרועות רובוטיות | שולט על מיקום מפרק דרך מאמץ (torque)     | effort_controllers/JointPositionController   |
| רובוטים פשוטים  | שולט על מיקום מפרק ישירות                | position_controllers/JointPositionController |
| גלגלים ומנועים  | שולט על מהירות מפרק                      | velocity_controllers/JointVelocityController |
| דינמיקה מתקדמת  | שולט על מהירות דרך מומנט                 | effort_controllers/JointVelocityController   |
| רובוטים ניידים  | שולט על רובוט עם גלגלים                  | diff_drive_controller/DiffDriveController    |
| נדרש כמעט תמיד  | רק מפרסם את המיקום/מהירות/מאמץ של joints | joint_state_controller/JointStateController  |

במקרה שלנו השתמשנו ב:

- joint\_state\_controller/JointStateController – משוב. ניתן לקבלו כמהירות, מיקום, או מומנט
- diff\_drive\_controller/DiffDriveController -הגדרת המנוע כהנעת diff\_drive

הרחבה: הגדרת פרמטרים נוספים

בדוגמא הבאה נציג כיצד ניתן להגדיר פרמטרים כלליים ופרמטרי PID:

```
# פרמטרים כלליים
pi: 3.14159
kind_of_robot: "arm_bot"
wheel_radius: 0.05
max_speed: 1.2 # מטר לשנייה

# לכל מפרק PID קבועי
joint1_controller:
  pid: {p: 10.0, i: 0.1, d: 0.01}

joint2_controller:
  pid: {p: 15.0, i: 0.2, d: 0.02}
```

במקטע קוד הבא ניתן לראות את האופן בו ניתן להשתמש בפרמטרים אלו (באמצעות פקודה "rospy.get\_param")

```
pi = rospy.get_param('pi')
robot_kind = rospy.get_param('kind_of_robot')
wheel_radius = rospy.get_param('wheel_radius')
max_speed = rospy.get_param('max_speed')

# joint1 של PID קבלת קבועי
pid_joint1 = rospy.get_param('joint1_controller/pid')
p1, i1, d1 = pid_joint1['p'], pid_joint1['i'], pid_joint1['d']
```

---

יצירת NODE (קובץ PY): לכתיבת לוגיקה

לאחר שהגדרנו את המודול המכני של הרובוט, חיישנים מנועים וכדו' כעת נעסוק בכתיבת לוגיקה במערכת.

את הלוגיקה נכתוב בקובץ מסוג py (כלומר python ניתן לכתוב גם ב++ אך לזה יש כללים קצת אחרים).

באופן עקרוני, בניגוד לקוד רגיל, בros כל "לוגיקה" צריכה להיות משוייכת ליישות מסויימת, ולכן נגדיר בתחילת הקוד node שתחתיו תופעל כל לוגיקה.

לפני שנתחיל עם הקוד, נבצע מספר פעולות טכניות:

- ניצור תיקיה בשם script ושם ניצור קובץ מסוג py ה(my\_ws/src/my\_pkg/script/my\_logic.py)
- לאחר מכן, נדרש "להפעיל" (execute) את הקובץ.

```
chmod +x ~/my_ws/src/my_pkg/scripts/my_logic.py
```

- ולסיום נעשה "build":

```
cd ~/my_ws  
source devel/setup.bash
```

לאחר מכן, כל שינוי בגוף הקובץ עצמו לא דורש מעבר לשמירת הקובץ.

כעת נעסוק בקוד עצמו:

תחילה, נציג את המבנה הבסיסי של node סטנדרטי שנכתב בpython. לאחר מכן נציג את הקוד הרלוונטי אלינו, כדי שנלמד

```
#!/usr/bin/env python

import rospy

def main():
    # Node-אתחול ה
    rospy.init_node('hello_node')

    # הדפסת הודעה אחת ללוג
    rospy.loginfo("hello_world")

if __name__ == '__main__':
    try:
        main()
    except rospy.ROSInterruptException:
        pass
```

את אופן כתיבת תכולת הקוד.

הסבר קצר על מבנה הקוד הבסיסי:

| Code                                                                                                               | desc                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| #!/usr/bin/env python                                                                                              | שורת קוד חובה שנועדה לאפשר להריץ את הקובץ כסקריפט                                                                                |
| import rospy                                                                                                       | יבוא ספריות רלוונטיות<br>rospy – יהיה חובה.<br>יבוא מבנה הודעות יוצג כ: <code>from mmm import nnnn</code>                        |
| rospy.init_node('hello_node')                                                                                      | הגדרת node: כלומר, ישירות המסוגלת לקבל/לשלוח הודעות, לבצע לוגיקה וכדו'                                                           |
| def main():                                                                                                        | <b>הפונקציה הראשית</b> שבה מתבצע כל הלוגיקה.                                                                                     |
| <pre>if __name__ == '__main__':     try:         main()     except rospy.ROSInterruptException:         pass</pre> | מבנה סטנדרטי שמריץ את <b>התוכנית</b> (מבלי להכנס לפרטים).<br>במקרה זה, ביצוע הלוגיקה מתבצע בעת ריצת הפונקציה <code>main()</code> |

לאחר הבנת המבנה הבסיסי של node נעסוק בכתיבת הלוגיקה:



```
#!/usr/bin/env python
import rospy
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import Twist

class GoUntilWall:
    def __init__(self):
        rospy.init_node('go_until_wall')
        self.cmd_pub = rospy.Publisher('/cmd_vel', Twist, queue_size=10)
        rospy.Subscriber('/scan', LaserScan, self.scan_callback)

        self.cmd = Twist()
        self.rate = rospy.Rate(10)
        self.distance = float('inf')
        self.state = "go"
        self.start_turn_time = None

    def scan_callback(self, msg):
        center_index = len(msg.ranges) // 2
        self.distance = msg.ranges[center_index]

    def spin(self):
        while not rospy.is_shutdown():
            if self.state == "go":
                if self.distance > 0.3:
                    self.cmd.linear.x = 0.2
                    self.cmd.angular.z = 0.0
                else:
                    self.cmd.linear.x = 0.0
                    self.state = "turn"
                    self.start_turn_time = rospy.Time.now()

            elif self.state == "turn":
                elapsed = rospy.Time.now() - self.start_turn_time
                if elapsed.to_sec() < 1.57: # בערך רבע סיבוב (90°) במהירות זוויתית של 1 רד/שנ
                    self.cmd.linear.x = 0.0
                    self.cmd.angular.z = -1.0 # סיבוב ימינה
                else:
                    self.cmd.angular.z = 0.0
                    self.state = "done"
                    rospy.loginfo("Finished rotation")

            else:
                self.cmd.linear.x = 0.0
                self.cmd.angular.z = 0.0

            self.cmd_pub.publish(self.cmd)
            self.rate.sleep()

if __name__ == "__main__":
    GoUntilWall().spin()
```

נסביר את תכולת הקוד:

| Code                                                                                                                                                                                                                                                                                                                                                                                                       | desc                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>import rospy from sensor_msgs.msg import LaserScan from geometry_msgs.msg import Twist</pre>                                                                                                                                                                                                                                                                                                          | <p><b>ייבוא:</b></p> <ul style="list-style-type: none"> <li>ספרייה לכתיבת קוד python בros</li> <li>מבנה msg יעודי לסורק לייזר</li> <li>מבנה msg יעודי למרחק זווית</li> </ul>                                                                                                                             |
| <pre>class GoUntilWall:     def __init__(self):         rospy.init_node('go_until_wall')         self.cmd_pub = rospy.Publisher('/cmd_vel', Twist, queue_size=10)         rospy.Subscriber('/scan', LaserScan, self.scan_callback)          self.cmd = Twist()         self.rate = rospy.Rate(10)         self.distance = float('inf')         self.state = "go"         self.start_turn_time = None</pre> | <p><b>יצירת class</b></p> <ul style="list-style-type: none"> <li>הפעלת פונק' אתחול</li> <li>יצירת node</li> <li>יצירת publiser – מפרסם הודעה בtopic בשם /cmd_vel המיועד לשליחת פקודה למנוע diff_drive</li> <li>יצירת subscriber – קורא הודעה מהסורק לייזר</li> <li>הגדרה + אתחול יתר הפרמטרים</li> </ul> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>def scan_callback(self, msg):     center_index = len(msg.ranges) // 2     self.distance = msg.ranges[center_index]</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | <p>פונקציה שנועדה לשמור את הערך של המרחק בכיוון מרכז הרובוט (הממוקם במרכז קשת הסריקה)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <pre>def spin(self):     while not rospy.is_shutdown():         if self.state == "go":             if self.distance &gt; 0.3:                 self.cmd.linear.x = 0.2                 self.cmd.angular.z = 0.0             else:                 self.cmd.linear.x = 0.0                 self.state = "turn"                 self.start_turn_time = rospy.Time.now()          elif self.state == "turn":             elapsed = rospy.Time.now() - self.start_turn_time             if elapsed.to_sec() &lt; 1.57: # זוויתית של 1 רד/שנ                 self.cmd.linear.x = 0.0                 self.cmd.angular.z = -1.0 # סיבוב ימינה             else:                 self.cmd.angular.z = 0.0                 self.state = "done"                 rospy.loginfo("Finished rotation")          else:             self.cmd.linear.x = 0.0             self.cmd.angular.z = 0.0          self.cmd_pub.publish(self.cmd)         self.rate.sleep()</pre> | <p>פונקציה התנועה, בה אנו רואים:</p> <ul style="list-style-type: none"> <li>• פקודה לתנועת הרובוט: <ul style="list-style-type: none"> <li>◦ <code>cmd.linear.x</code> - תנועה בכיוון x</li> <li>◦ <code>cmd.angular.z</code> - תנועה סביב ציר z</li> </ul> </li> <li>• קבלת המרחק בכיוון הנסיעה של הרובוט: <ul style="list-style-type: none"> <li>◦ <code>self.distance</code> (שיוצר בפונקציה הקודמת)</li> </ul> </li> </ul> <p>באופן עקרוני תנועת הרובוט הינה:</p> <ul style="list-style-type: none"> <li>• נסיעה בכיוון x במהירות 0.2 – עד למרחק 0.3</li> <li>• עצירה</li> <li>• סיבוב 90 מעלות ימינה</li> <li>• עצירה</li> </ul> <p>בפועל, הפקודה למנוע תתבצע בשורה הזו: הפעלת <code>publisher</code> לפי הערכים שניתנו למעלה</p> |
| <pre>if __name__ == "__main__":     GoUntilWall().spin()</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <p>הפעלת הפונקציה הראשית</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

## קובץ LAUNCH: הרצה מרוכזת

קובץ launch הוא קובץ הרצה שנועד להריץ באמצעות פקודה אחת מספר פעולות, בסדר מסויים.

יתרונו מקבל משנה תוקף כאשר:

- נרצה להריץ מספר קבצים בזה אחר זה
- טעינת קובץ יחד עם טעינת הפרמטרים אשר ישפיעו אופן פעולתו

כפי שצוין, קובץ זה נועד להריץ מספר סוגי פעולות: הפעלת הסימולציה (world), טעינה ופרסום של מודל הרובוט (urdf), טעינת פרמטרים כללים, בקרים וקבועי בקרה (yaml) ועוד. בפרק זה נלמד את אופן ההגדרה של כל אחד מסוגי הפעולות הללו. כדי לתת תמונה מלאה, נתאר את הפעולות האפשריות ואופן הגדרתן.

כל הפעולות בקובץ זה מתבצעים ע"י שלושת הפעולות הבאות: טעינת פרמטרים, הרצת node, וטעינת קובץ launch. כעת נציג את אופן התחביר שלהם:

### טעינת פרמטרים:

```
<param name="robot_description" command="$(find xacro)/xacro $(find my_package)/urdf/my_robot.urdf.xacro" />
```

סמל: param

שדות רלוונטיים:

- **name** – שם הפרמטר שנרצה להגדיר. לצורך שימוש פינימי ב-ros
- **command** – פקודה שתורץ ותכניס את הערכים לתוך הפרמטר (כאן הפקודה – יצירת קובץ urdf דינמי מ-xacro)
- **(value)** – יופיע במקום command כאשר יש ערך סופי וברור

### הרצת node

```
<node name="my_control_node"
      pkg="my_robot_pkg"
      type="go_to_position.py"
      output="screen"/>
```

סמל: node

שדות רלוונטיים:

- **pkg** – החבילה בה ממוקדם node.
- **type** – שם ה-node.
- **(arg)** – פרמטרים שיעברו להרצה.

## הרצת קובץ launch אחר

```
<include file="$(find gazebo_ros)/launch/empty_world.launch">
  <arg name="paused" value="false"/>
</include>
```

סמל: include

שדות רלוונטים:

- file – הניתוב של קובץ הlaunch
- (arg) – טעינת פרמטרים רלוונטים (שמוגדרים כבר באותו קובץ).

כללים נוספים:

- הרצת controller מתבצעת כהרצת node כאשר בשדה ה arg יופיעו שמות הבקרים

```
<node name="controller_spawner" pkg="controller_manager"
      type="spawner" args="controller1 controller2 ..." />
```

- צורת טעינת הסימולציה מתבצע באופן הבא:
  - טעינת קובץ עולם (או קובץ world או כקובץ launch של עולם קיים בgazebo)
  - יצירת פרמטר שיכיל את מודל הרובוט
  - הרצת node המשויך לחבילה gazebo\_ros כאשר מודל הרובוט מגיע כפרמטר

```
<launch>
  <include file="$(find gazebo_ros)/launch/empty_world.launch"/>

  <param name="robot_description" command="$(find xacro)/xacro '$(find my_robot_pkg)/urdf/my_robot.xacro'" />

  <node name="spawn_urdf" pkg="gazebo_ros" type="spawn_model"
        args="-param robot_description -urdf -model my_robot"
        output="screen"/>
</launch>
```

כעת נציג את קובץ launch בפרויקט שלנו

```

<launch>
  <!-- עם עולם ריק Gazebo אתחול -->
  <include file="$(find gazebo_ros)/launch/empty_world.launch">
    <arg name="paused" value="false"/>
    <arg name="use_sim_time" value="true"/>
    <arg name="gui" value="true"/>
    <arg name="headless" value="false"/>
  </include>

  <!-- URDF-טעינת פרמטרים של ה -->
  <param name="robot_description" command="$(find xacro)/xacro $(find my_robot_description)/urdf/my_robot.urdf.xacro" />

  <!-- Gazebo-של הרובוט ל Spawn -->
  <node name="spawn_urdf" pkg="gazebo_ros" type="spawn_model" output="screen"
    args="-param robot_description -urdf -model my_robot -x 0 -y 0 -z 0.1"/>

  <!-- controller manager טעינת -->
  <node name="controller_spawner" pkg="controller_manager" type="spawner" respawn="false" output="screen"
    args="joint_state_controller
          left_wheel_controller
          right_wheel_controller
          lidar_joint_controller"/>

  <!-- joint states מפרסם את -->
  <node name="robot_state_publisher" pkg="robot_state_publisher" type="robot_state_publisher" output="screen"/>

  <!-- לצפייה בלוגים (לא חובה) rqt_console -->
  <!-- <node pkg="rqt_console" type="rqt_console" name="rqt_console" /> -->

</launch>

```

הסבר על הקוד:

| code                                                                                                                                                                                                                                                                                                                      | desc                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> &lt;launch&gt; ... &lt;/launch&gt; </pre>                                                                                                                                                                                                                                                                           | מסגרת הקוד                                                                                                                                                                                                                             |
| <pre> &lt;!-- עם עולם ריק Gazebo אתחול --&gt; &lt;include file="\$(find gazebo_ros)/launch/empty_world.launch"&gt;   &lt;arg name="paused" value="false"/&gt;   &lt;arg name="use_sim_time" value="true"/&gt;   &lt;arg name="gui" value="true"/&gt;   &lt;arg name="headless" value="false"/&gt; &lt;/include&gt; </pre> | <p>טעינת עולם בgazebo.</p> <p>כאן נטען "עולם ריק", מתוך מספר אופציות מובנות של gazebo. כמובן שניתן לטעון עולם שיוצר על ידנו טעינת פרמטרים (די אינטואיטיבי). ניתן לראות באינטרנט את יתר הפרמטרים. מה שלא הוגדר, יקבל ערך ברירת מחדל</p> |
| <pre> &lt;!-- URDF-טעינת פרמטרים של ה --&gt; &lt;param name="robot_description" command="\$(find xacro)/xacro   \$(find my_robot_description)/urdf/my_robot.urdf.xacro" /&gt; </pre>                                                                                                                                      | <p>טעינת המודל (my_robot.urdf) לפרמטר ביניים (robot_description)</p>                                                                                                                                                                   |
| <pre> &lt;!-- Gazebo-של הרובוט ל Spawn --&gt; &lt;node name="spawn_urdf" pkg="gazebo_ros" type="spawn_model" output="screen"   args="-param robot_description -urdf -model my_robot -x 0 -y 0 -z 0.1"/&gt; </pre>                                                                                                         | <p>“זריקת” המודל” המוצג כפרמטר לgazebo באמצעות:</p> <ul style="list-style-type: none"> <li>יצירת node</li> <li>החבילה בgazebo: gazebo_ros</li> <li>סוג הפעולה: “זריקת מודל”</li> <li>פרמטרים רלוונטים</li> </ul>                       |

|                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> &lt;!-- טעינת controller manager --&gt; &lt;node name="controller_spawner" pkg="controller_manager"       type="spawner" respawn="false" output="screen"       args="joint_state_controller             left_wheel_controller             right_wheel_controller             lidar_joint_controller"/&gt; </pre> | <p><b>טעינת בקרים וחיישנים<br/>באמצעות:</b></p> <ul style="list-style-type: none"> <li>• יצירת node</li> <li>• החבילה gazebo: Controller_manager</li> <li>• סוג הפעולה: "זריקה" כללית</li> <li>• פרמטרים רלוונטים: הבקרים השונים והחיישן</li> </ul> |
| <pre> &lt;!-- joint states מפרסם את --&gt; &lt;node name="robot_state_publisher" pkg="robot_state_publisher"       type="robot_state_publisher" output="screen"/&gt; </pre>                                                                                                                                            | <p><b>טעינת משובים<br/>באמצעות:</b></p> <ul style="list-style-type: none"> <li>• יצירת node</li> <li>• החבילה gazebo: robot_state_publisher</li> <li>• סוג הפעולה: פרסום מיקומים</li> <li>• פרמטרים רלוונטים: אין</li> </ul>                        |

תקשורת:

בROS יש מספר צורות כיצד להעביר מידע בין ה-nodes השונים:

- topic – העברת מידע באופן חד כיווני. השולח (publisher) מפרסם מידע "לענן" בפורמט מוגדר, ואילו המקבל (subscriber) קורא אותו. התהליך לא דורש תזמון כלשהו. לדוגמא: חיישן מרחק lidar מפרסם מידע על המרחק ממנו והלאה, הפורמט של המידע: scan. כאשר node מסוים מעוניין במידע זה הוא קורא אותו.
- service – העברת מידע בצורה של שאילתא. ה-client שולח שאילתא (בהתאם לפורמט) ואז ה-server מחזיר לו את ה-datan. לדוגמא: לאחר הגדרת פורמט בקובץ msg ה-client שולח 2 מספרים, ובתגובה ה-server מחזיר לו את מכללתם.
- action – העברת מידע בצורה רציפה. לדוגמא: הגדרת מסלול נסיעה לרובוט. נושא זה לא יורחב במסגרת מדריך זה.

פרקטיקה:

TOPIC – העברת מידע באופן חד כיווני.

נעת נביא דוגמא לאופן העברת המידע:

הNode המפרסם (publisher):

מפרסם כל 1 שניה את הטקסט "hello from talker"

```
#!/usr/bin/env python
import rospy
from std_msgs.msg import String

rospy.init_node('talker')
pub = rospy.Publisher('chatter', String, queue_size=10)
rate = rospy.Rate(1)

while not rospy.is_shutdown():
    pub.publish("Hello from talker!")
    rate.sleep()
```

דגשים:

| פעולה                                                                                                                                                                                           | syntax                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| ייבוא סוג ההודעות                                                                                                                                                                               | <code>from std_msgs.msg import String</code>                                                              |
| הגדרת המשתנה pub כ"מפרסם". הגדרות רלוונטיות: <ul style="list-style-type: none"><li>• שם ההודעה "chatter"</li><li>• סוג פורמט ההודעה string (שייבאה מקודם ב <code>from..import</code>)</li></ul> | <code>pub = rospy.Publisher('chatter', String, queue_size=10)</code><br><code>rate = rospy.Rate(1)</code> |
| פרסום                                                                                                                                                                                           | <code>pub.publish("Hello from talker!")</code>                                                            |

## הNode המקבל (subscriber):

קורא את ההודעה בשם "chatter" מסוג string ומפרסם אותה בlog

```
#!/usr/bin/env python
import rospy
from std_msgs.msg import String

def callback(msg):
    rospy.loginfo("Received: %s", msg.data)

rospy.init_node('listener')
sub = rospy.Subscriber('chatter', String, callback)
rospy.spin()
```

## דגשים:

| פעולה                                                                                                                                                                                                     | syntax                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| ייבוא סוג ההודעות                                                                                                                                                                                         | <pre>from std_msgs.msg import String</pre>                                |
| הגדרת המשתנה sub כ"קורא". הגדרות רלוונטיות: <ul style="list-style-type: none"><li>שם המפרסם</li><li>סוג פורמט ההודעה</li><li>הפונקציה שתקבל את המידע (callback)</li></ul> לאחר ההגדרה – sub יקבל את המידע | <pre>sub = rospy.Subscriber('chatter', String, callback)</pre>            |
| הפונקציה callback פרסום ההודעה בlog                                                                                                                                                                       | <pre>def callback(msg):     rospy.loginfo("Received: %s", msg.data)</pre> |



## SERVICE – העברת מידע בצורה של שאילתא.

ראשית נגדיר את פורמט ההודעה:

- נייצר תיקיה srv ובתוכה ניצור קובץ מסוג srv: `AddTwoInts.srv`
- נגדיר את פורמט ההודעה באופן הבא:

```
int64 a
int64 b
---
int64 sum
```

כלומר: הclient שולח את a,b מסוג int64 והserver יחזיר לו את sum (גם מסוג int64)

ונבצע: `catkin_make`

**Client:**

```
#!/usr/bin/env python
import rospy
from my_pkg.srv import AddTwoInts

rospy.init_node('add_client')
rospy.wait_for_service('add_two_ints')
add = rospy.ServiceProxy('add_two_ints', AddTwoInts)
resp = add(5, 7)
rospy.loginfo("Sum: %s", resp.sum)
```

**דגשים:**

| פעולה                                                                                                                                                      | syntax                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| יבוא סוג ההודעות (שם הpkg, ושם קובץ srv)                                                                                                                   | <pre>from my_pkg.srv import AddTwoInts</pre>                                                           |
| הגדרת שם service שממנו נקבל data<br>הגדרת המשתנה add כclient. עם ההגדרות: <ul style="list-style-type: none"><li>• שם server</li><li>• סוג ההודעה</li></ul> | <pre>rospy.wait_for_service('add_two_ints') add = rospy.ServiceProxy('add_two_ints', AddTwoInts)</pre> |
| שליחת השאילתה. (המידע עובר באותו רגע)<br>הדפסתו לlog                                                                                                       | <pre>resp = add(5, 7) rospy.loginfo("Sum: %s", resp.sum)</pre>                                         |

Server:

```
#!/usr/bin/env python
import rospy
from my_pkg.srv import AddTwoInts, AddTwoIntsResponse

def handle_add(req):
    return AddTwoIntsResponse(req.a + req.b)

rospy.init_node('add_server')
srv = rospy.Service('add_two_ints', AddTwoInts, handle_add)
rospy.spin()
```

| פעולה                                                                                                                              | syntax                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| יבוא סוג ההודעות                                                                                                                   |                                                                              |
| <ul style="list-style-type: none"><li>מבנה ההודעת srv: (שם pkg, ושם קובץ srv)</li><li>הודעת החזר של server</li></ul>               | <pre>from my_pkg.srv import AddTwoInts, AddTwoIntsResponse</pre>             |
| הגדרת המשתנה srv כserver. עם ההגדרות:                                                                                              |                                                                              |
| <ul style="list-style-type: none"><li>שם server (ישומש גם בclient)</li><li>סוג ההודעה</li><li>הפונקציה שתופעל handle_add</li></ul> | <pre>srv = rospy.Service('add_two_ints', AddTwoInts, handle_add)</pre>       |
| הפונקציה handle_add                                                                                                                |                                                                              |
| אשר מחזירה (באמצעות "הודעת החזר") a+b                                                                                              | <pre>def handle_add(req):     return AddTwoIntsResponse(req.a + req.b)</pre> |

עדכון קבצי PACKAGE.XML, CMAKELIST.TXT

קבצים אלו נועדו לצורך פעולת הbuild של package

## PACKAGE.XML

**תפקיד:** קובץ זה הוא לנהל את התיאור המטא-דטה של החבילה ROS (package). משתמש ב-package.xml כדי לדעת אילו חבילות דרושות בעת בנייה והרצה.

**מתי נדרש לעדכן ידנית:** נדרש לעדכן קובץ זה באופן ידני בכל פעם שנשתמש ב"תלותות" במערכת כדוגמת: rospy, sensor\_msgs וכדו' (בד"כ כשנייבא "חבילות" באמצעות import בראשית הnode, וכן בכל מיני בקרים, חיבור לgazebo וכדו'..)

### מבנה הקובץ:

```
home > matanya > myarm_ws > src > my_robot_arm > package.xml
2 <package format="2">
3   <!-- Use doc_depend for packages you need only for building documents
4   <!--   <doc_depend>doxygen</doc_depend> -->
50 <!--   <doc_depend>doxygen</doc_depend> -->
51 <buildtool_depend>catkin</buildtool_depend>
52
53 <build_depend>gazebo_ros</build_depend>
54 <build_depend>controller_manager</build_depend>
55 <build_depend>effort_controllers</build_depend>
56 <build_depend>joint_state_controller</build_depend>
57 <build_depend>joint_state_publisher</build_depend>
58 <build_depend>robot_state_publisher</build_depend>
59 <build_depend>rospy</build_depend>
60 <build_depend>sensor_msgs</build_depend>
61 <build_depend>std_msgs</build_depend>
62 <build_depend>urdf</build_depend>
63 <build_depend>xacro</build_depend>
64
65 <build_export_depend>gazebo_ros</build_export_depend>
66 <build_export_depend>controller_manager</build_export_depend>
67 <build_export_depend>effort_controllers</build_export_depend>
68 <build_export_depend>joint_state_controller</build_export_depend>
69 <build_export_depend>joint_state_publisher</build_export_depend>
70 <build_export_depend>robot_state_publisher</build_export_depend>
71 <build_export_depend>rospy</build_export_depend>
72 <build_export_depend>sensor_msgs</build_export_depend>
73 <build_export_depend>std_msgs</build_export_depend>
74 <build_export_depend>urdf</build_export_depend>
75 <build_export_depend>xacro</build_export_depend>
76
77
78 <exec_depend>gazebo_ros</exec_depend>
79 <exec_depend>controller_manager</exec_depend>
80 <exec_depend>effort_controllers</exec_depend>
81 <exec_depend>joint_state_controller</exec_depend>
82 <exec_depend>joint_state_publisher</exec_depend>
83 <exec_depend>robot_state_publisher</exec_depend>
84 <exec_depend>rospy</exec_depend>
85 <exec_depend>sensor_msgs</exec_depend>
86 <exec_depend>std_msgs</exec_depend>
87 <exec_depend>urdf</exec_depend>
88 <exec_depend>xacro</exec_depend>
89
```

נשים לב, שכל "תלות" תופיע בכל 3 השדות השונות.

---

## CMAKELISTS.TXT

**תפקיד:** קובץ הבנייה (build script) של CMake עבור החבילה. קובץ זה נועד להבטיח שהקוד מקומפל, שההודעות מיוצרות, ושה-libraries מקושרות נכון.

**מתי לעדכן ידנית:** באותם נסיבות שעדכנו את package.xml

### מבנה הקובץ:

```
## Find catkin macros and libraries
## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
## is used, also find other catkin packages
find_package(catkin REQUIRED COMPONENTS

  gazebo_ros
  controller_manager
  effort_controllers
  joint_state_controller
  joint_state_publisher
  robot_state_publisher
  rospy
  sensor_msgs
  std_msgs
  urdf
  xacro)
```

הערה חשובה: בנושא זה, וכן בכל נושא הקבצים והתקיות של בניה וההדרה של הפרויקט (ws) יש לא מעט מה להרחיב ולעיתים מתגלים באגים שנפתרים על ידי עדכון פרמטרים בקבצים אלו. מכיוון שאינני בקיא בהם, לא ארחיב במסגרת מדריך זה.

---

## הרצת המערכת מהTERMINAL

לאחר סיום פיתוח הפרויקט – נרצה להריץ את המערכת. פעולה זו יכולה להתבצע במספר אפשרויות כתלות בצורך:

- הרצת node צפציפי

```
roslaunch <package_name> <node_file.py>
```

- טעינת פרמטר

```
roslaunch <package_name> <node_file.py> --param /my_param 42
```

- שליחת פקודה (פרסום בtopic)

```
rostopic pub /<topic_name> <message_type> <data>
```

```
rostopic pub -1 /joint1_effort_controller/command std_msgs/Float64 "data: 0.5"
```

- הפעלת קובץ launch (שכאמור מריץ מספר פעולות)

```
roslaunch <package_name> <file_name.launch>
```

במדריך זה ניסינו להקיף את המסה הקריטית הרלוונטית למפתח המתחיל: החל מהבנה התאורטית דרך תיאור השלבים בפיתוח הפרויקט וכלה בהבנה המבנה ותוכן הקבצים והתיקיות בפרויקט. אני מאוד מקווה שמדריך זה היה לעזר וסייע בקידום הנגשת ros בכלל כניסה לעולם פיתוח הרובוטים בפרט.

חרף זאת, ישנם עוד מספר נושאים שטרם נגענו בהם ואותם הקורא ירחיב על פי צורך באופן עצמאי או שבעתיד נוסיף אותו למדריך בגרסה משודרגת יותר. הנושאים החסרים במדריך זה:

- כל קבצי ותיקיות ה"בניה וההדרה" של הפרויקט
- action
- הבדל בין ros1 ל ros2
- אימות ובדיקה של כל פעולה דרך הטרמינל
- נושא הניווט והמיפוי
- ממשק לכלי AI
- ממשק לכלים מתמטים כדוגמת matlab לניתוח הנתונים.